

Foundation Model for **1D NMR Metabolomics**

What we built, what we found, and the three things we got wrong

Shankararama Sharma

Group Meeting · August 2026

continues from 27 Feb 2026

9,670

spectra pretrained on
(was 2,146 in Feb)

5

evaluation targets
across 4 cohorts

15

numbered experiments
3 retractions

The goal — unchanged since February

Why a foundation model is the right idea for metabolomics

1

The problem

Metabolomics cohorts are tiny — 10s to 100s of samples. Deep networks need far more than that to train from scratch.

2

The idea

Pretrain once on thousands of UNLABELLED spectra with a self-supervised task. No labels needed — we have plenty of raw spectra.

3

The payoff

Transfer that representation to a small labelled cohort. The backbone already knows what NMR spectra look like.

4

Success criterion

It must beat classical ML (binned intensities + logistic regression) on small datasets. That is the bar.

Where we were on 27 February

The state of play at the last group meeting

What was working

- Masked Spectra Modelling trained end-to-end
- 2,146 CPMG serum spectra from MetaboLights
- Reconstruction was excellent — $\text{MSE} \approx 3 \times 10^{-5}$, $R^2 \approx 0.98$ at 25–35% masking
- Full preprocessing pipeline: TopSpin → align → water-suppress → dedupe → normalise
- Three evaluation routes sketched: embeddings+ML, classification head, ML-only

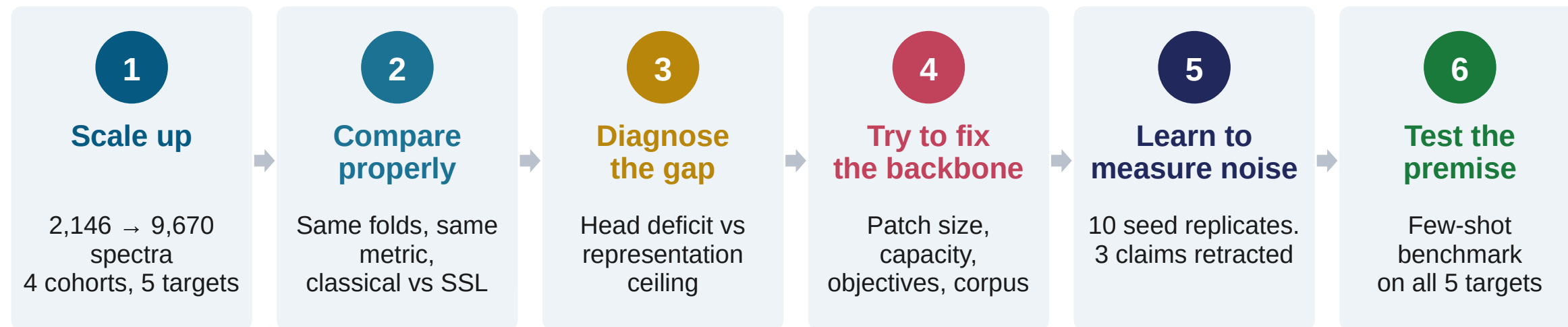
What was missing

- Only ONE test cohort (TBI, ~57 samples) — no way to tell general from idiosyncratic
- No head-to-head against classical ML on identical folds
- No control asking whether pretraining helped at all vs a random network
- No error bars anywhere — every arm was a single training run
- Reconstruction quality was our success metric. It turns out not to be one.

Most of the last five months went into fixing the right-hand column.

What happened since — the arc

Five months in one picture



The honest summary of the arc

Steps 1–3 worked and produced two real improvements. Step 4 failed on every axis we tried. Step 5 revealed that most of step 4's "results" were noise. Step 6 tested the project's core premise directly — and it did not hold.

Step 1 — more data, and real evaluation cohorts

February had one test set. Now there are five targets across four independent cohorts.

Pretraining corpus

2,146 → 9,670

unlabelled CPMG serum / plasma
spectra
MetaboLights + Metabolomics
Workbench
131,072 points each, deduplicated

Evaluation target	Size	Label	Protocol
Barth Syndrome	n = 37	Case / Control	LOOCV
MTBLS326	n = 42	Yes / No (IP3R)	LOOCV
MTBLS563	n = 113	3-class diagnosis	10-fold
BrC-T2D — cancer	n = 78	Cancer / No cancer	10-fold
BrC-T2D — diabetes	n = 78	Diabetes / No	10-fold

Two things to flag about the evaluation set

- Barth and MTBLS326 use LOOCV, so they have NO fold variance — no error bars. Differences of 0.02 there are one sample.
- MTBLS326's classical score is a perfect 1.000 on n=42. It clears a permutation null, but we have not yet checked run-order / batch confounding. Until we do, it should not count as evidence.

A detour worth mentioning — four preprocessing versions

We found a real bug. Then we found out it did not matter.

v1

Water suppression only
points 62,500–68,000 $\rightarrow$ 0



v2

EDTA window skipped
(left largely untouched)



v3

Uniform magnitude-based
EDTA suppression



v4

EDTA cutoff FIXED to the
baseline-to-peak midpoint

EDTA is a real contaminant problem

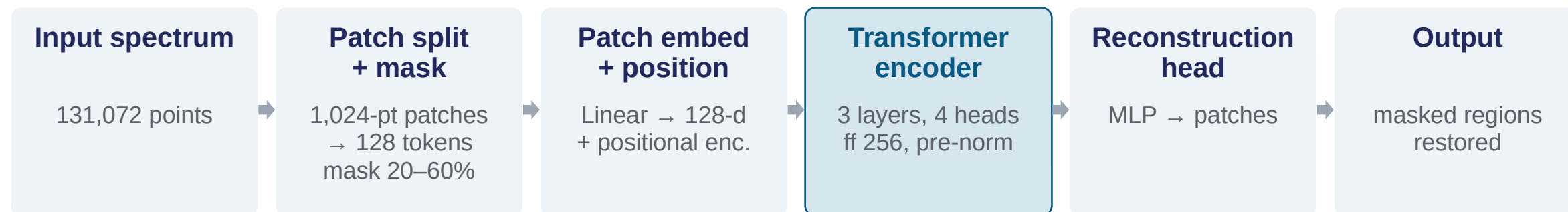
- Blood-collection tubes leave EDTA peaks that can dominate a spectrum and swamp the row normalisation.
- v3 $\rightarrow$ v4 fixed a genuine cutoff bug: the suppression was removing too much of the surrounding signal.
- 7 rows repo-wide still retain a dominant EDTA peak — documented, not hidden.

...but it does not matter downstream

- We spent two whole experiments trying to explain a v3-vs-v4 downstream difference.
- With 5 seeds per corpus that difference is -0.014 ± 0.022 — indistinguishable from zero (slide 19).
- Lesson: fix the preprocessing because it is CORRECT, not because you measured a downstream gain from one run.

The backbone, and the three objectives we tried

Architecture is unchanged from February; two more pretext tasks were added



1.89 M parameters · patch 1024 · d_model 128 · best val reconstruction 9.3×10^{-5}

Masked (MSM)

Hide 20–60% of patches,
reconstruct them.

The one that works

Jigsaw

Shuffle spectral bins, predict the
original order.

No better than random init

Joint

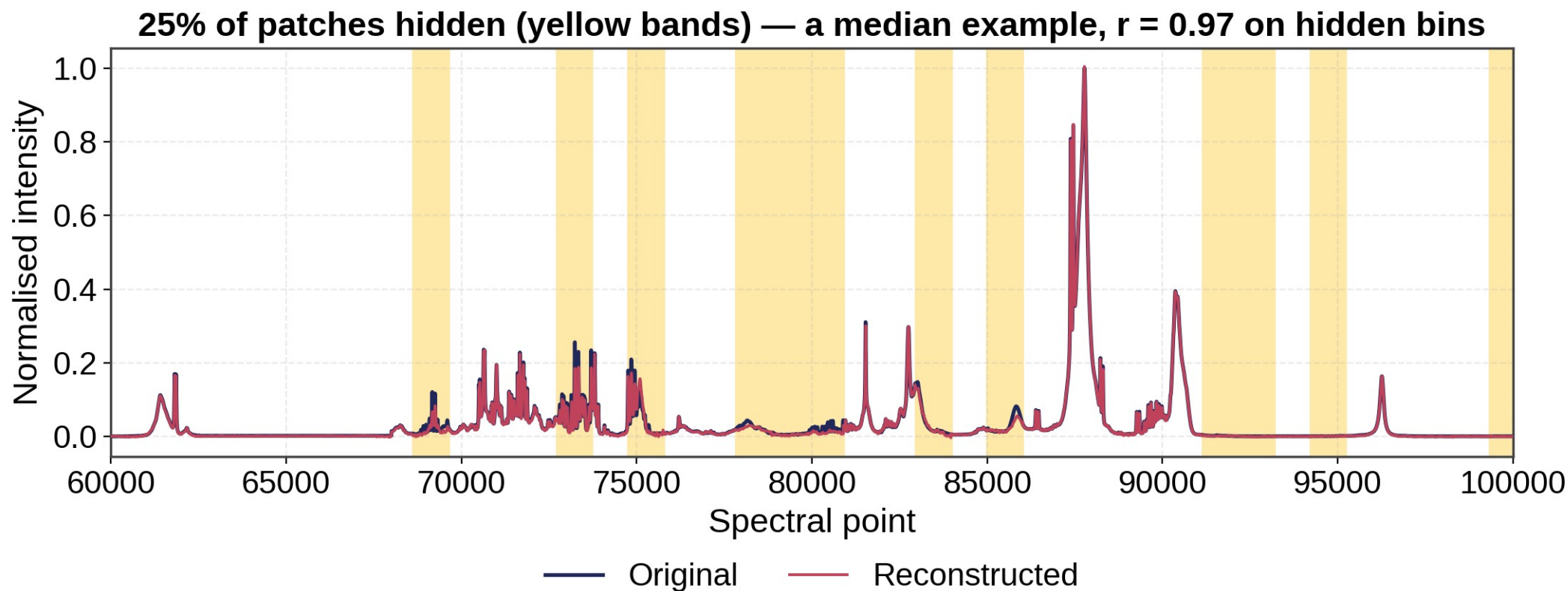
Masked + jigsaw together, shared
encoder.

Actively harmful

(the verdicts come from the random-init control on slide 14)

The pretext task itself works very well

Re-measured from February — and this is exactly where the trap was



Across 50 spectra: $r = 0.940 \pm 0.090$ on the hidden bins only

The trap: the pretext task is genuinely solved well — and it still tells you almost nothing. (February's $r = 0.999$ was whole-spectrum, which includes the 75% of bins the model is handed.) Across five backbones BETTER reconstruction went with WORSE transfer, so we never select a checkpoint, architecture or epoch on reconstruction loss.

How we compare now

The protocol that made everything after this interpretable

Identical folds

Both tracks see exactly the same splits — LOOCV for Barth / MTBLS326, stratified 10-fold (seed 42) for the rest.

Identical metric

Balanced accuracy everywhere, because several targets are imbalanced (Barth is 14 / 23).

Both classifiers linear

Classical = StandardScaler → LogReg. SSL = frozen embedding → LogReg probe. So a difference is about the FEATURES, not the classifier.

The validation gate

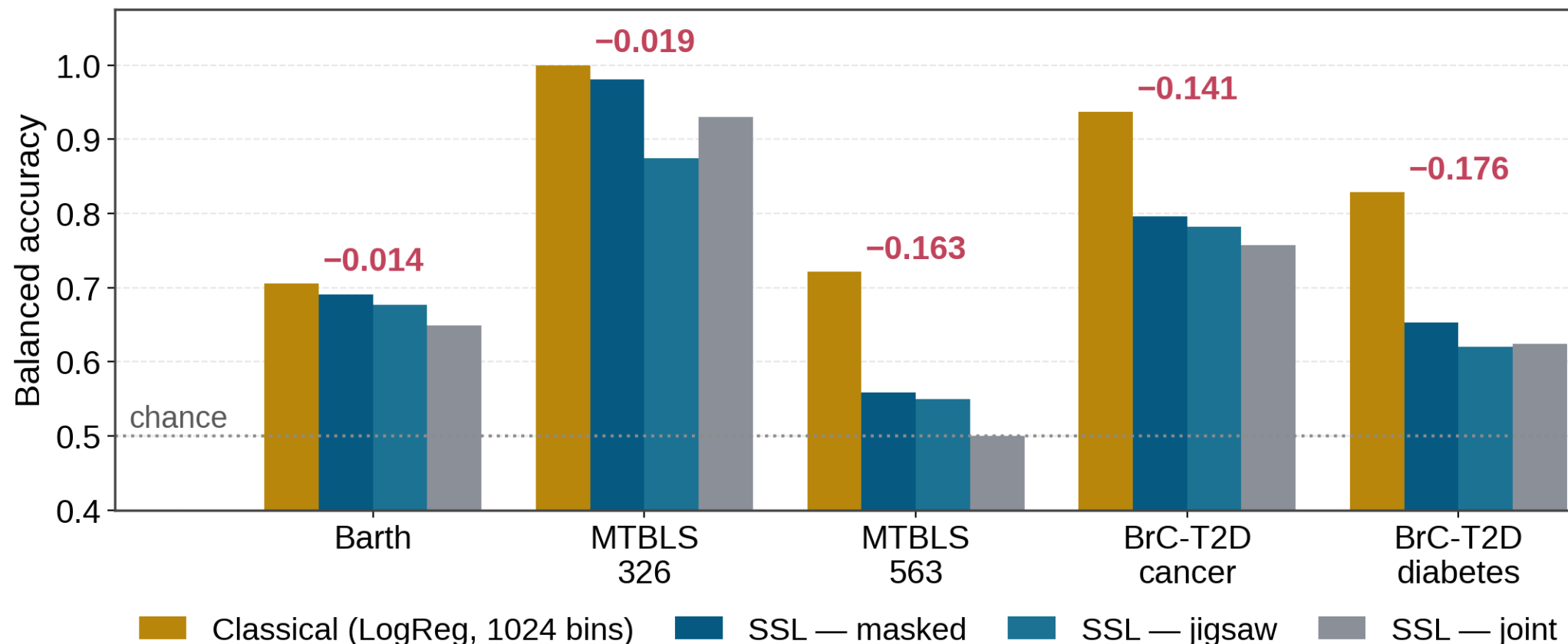
Before drawing any conclusion, the harness had to reproduce the committed classical numbers to 6 decimal places on all five targets:

BrC-T2D cancer	0.936842
BrC-T2D diabetes	0.828877
MTBLS563	0.720785
MTBLS326	1.000000
Barth	0.704969

Without this, none of the comparisons would mean anything.

Step 2 result — classical ML beat every SSL family

Balanced accuracy, v4 data, best mode per family, identical folds



Consistent ordering: masked > jigsaw ≈ joint. Two of the gaps are large (0.14–0.18). Barth and MTBLS326 are effectively ties — half a sample and one sample respectively.

The gap is not one problem — it is two

Run the SAME logistic regression on the frozen SSL embedding to separate them

Head-fitting deficit

The SSL head is a linear layer trained by Adam for ~50 epochs on a handful of samples. LogReg on the IDENTICAL features fits the same linear map properly (L-BFGS to convergence, explicit L2, standardised inputs).

Representation ceiling

Whatever survives the better classifier is the embedding's own limit — information the backbone never encoded, which 1,024-bin integrated areas do carry.

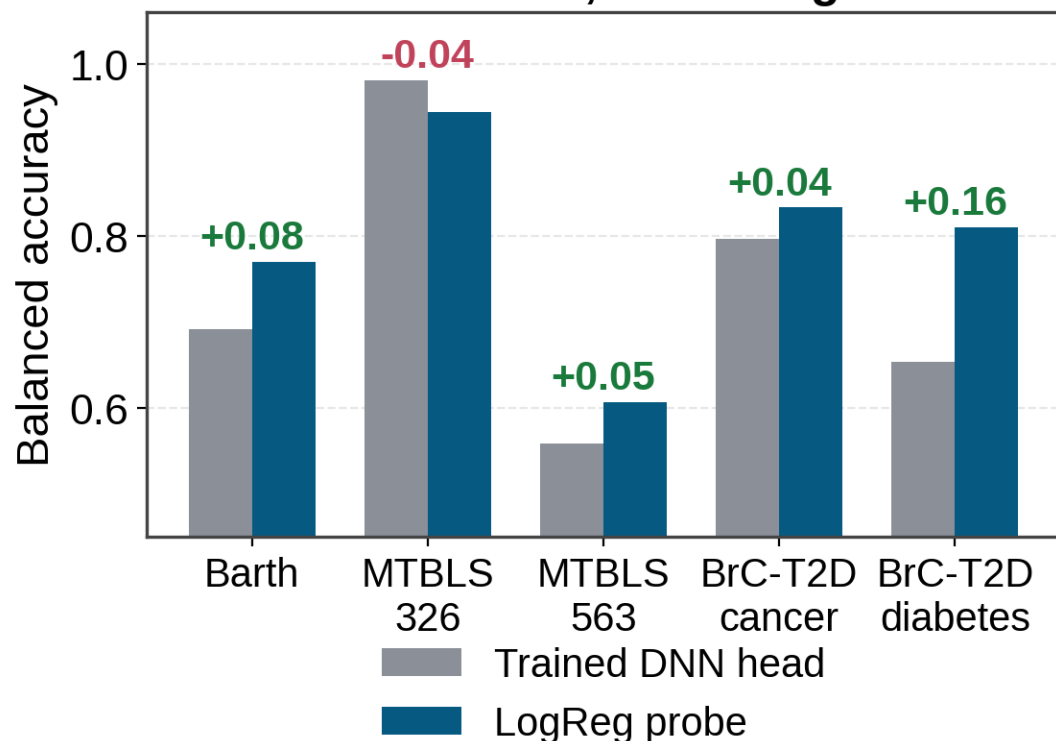
Target	SSL head	LogReg on same embedding	LogReg @ 1024 bins	Head deficit	Representation ceiling
BrC-T2D cancer	0.796	0.833	0.937	0.037	0.104
BrC-T2D diabetes	0.653	0.810	0.829	0.157	0.019
MTBLS563	0.558	0.607	0.721	0.049	0.114
Barth	0.691	0.770	0.705	0.079	-0.065
MTBLS326	0.981	0.944	1.000	-0.037	0.056

Diabetes is 89% a head problem — the embedding already supports 0.810. Cancer and MTBLS563 are mostly representation. Barth's embedding actually BEATS binned features (0.770 vs 0.705); the head was wasting it.

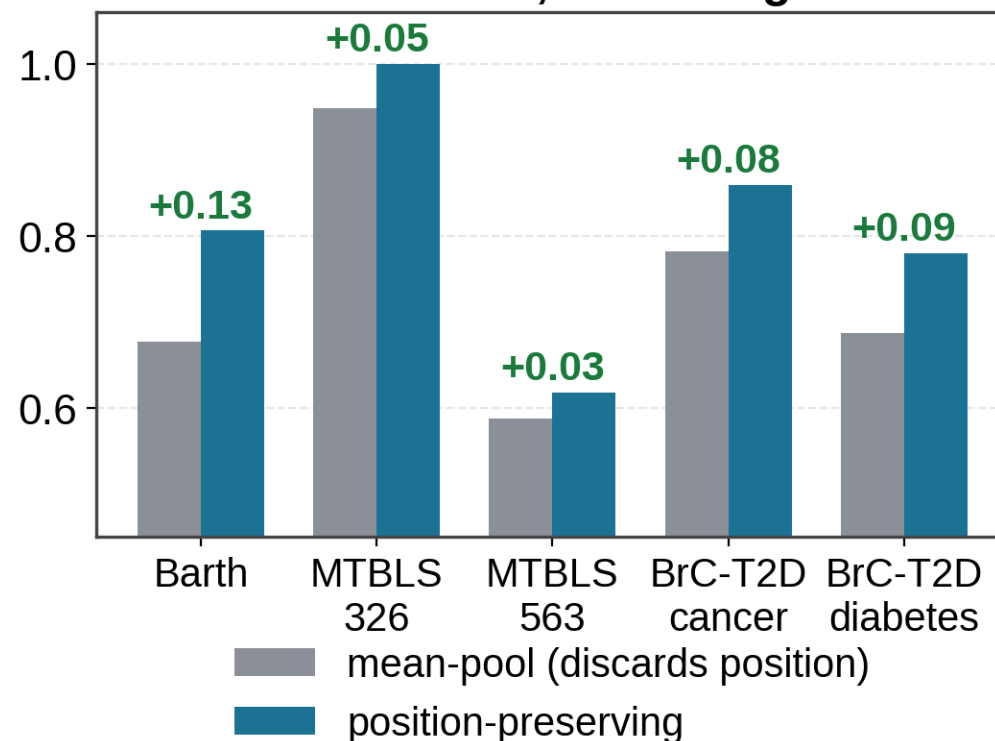
Two fixes that cost zero GPU time

Both are PAIRED comparisons — same frozen checkpoint, one thing changed. Both still stand today.

Win 1 — the head was underfit
+0.120 mean, 5 of 5 targets



Win 2 — pooling discarded position
+0.03...+0.13, 5 of 5 targets



Why these two survived everything that came later: they vary a transform on a FIXED checkpoint, so they carry no pretraining run-to-run variance at all. That distinction turns out to be the whole story of this project.

Where those two fixes left us

SSL vs classical, after the head and pooling fixes — no retraining

Target	Reported Feb-style head	Probe + pooling fix	Classical LogReg	vs classical	Verdict
Barth	0.691	0.806	0.705	+0.101	SSL WINS
MTBLS326	0.981	1.000	1.000	0.000	tie
BrC-T2D cancer	0.796	0.859	0.937	-0.078	classical
BrC-T2D diabetes	0.653	0.783	0.829	-0.046	classical
MTBLS563	0.558	0.621	0.721	-0.100	classical

+0.078

mean improvement over the
February-style numbers

0 → 1

wins against classical
(was 0 wins / 5 losses)

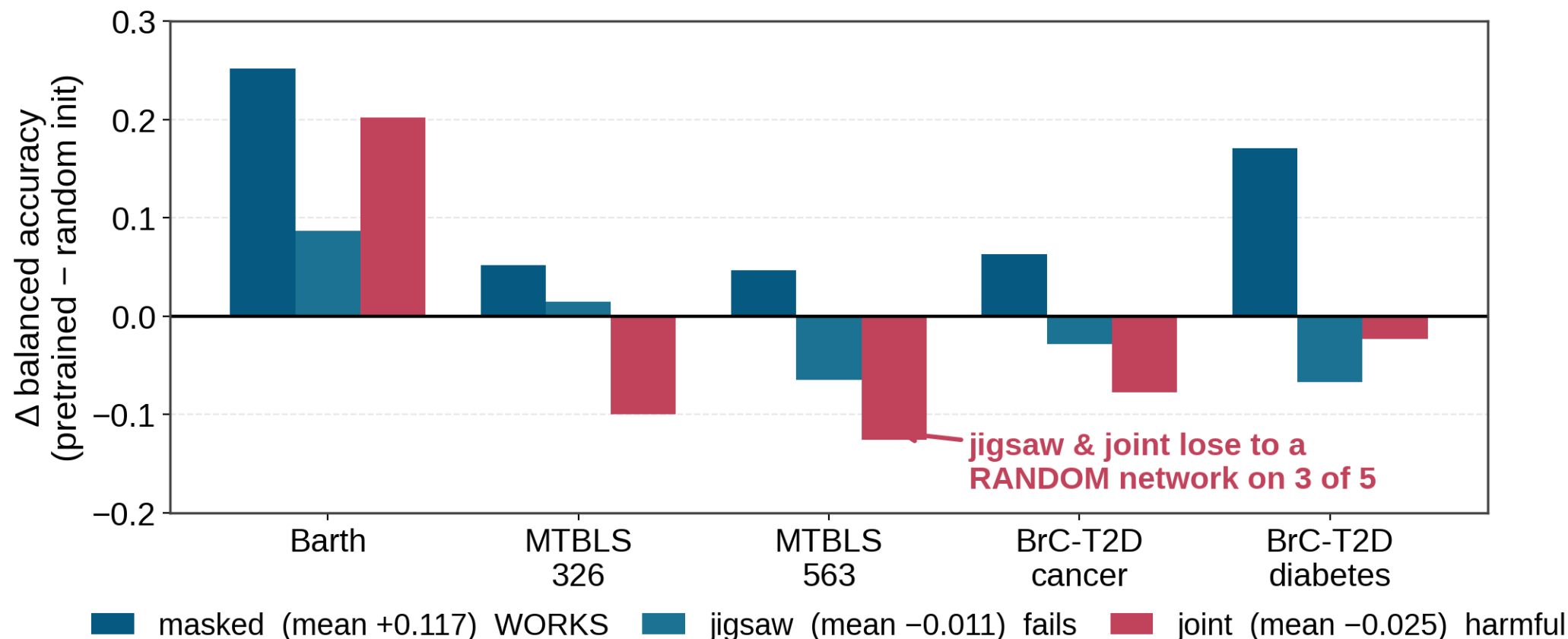
0 h

GPU time spent —
both fixes are post-hoc

Record moves from 0 wins / 0 ties / 5 losses → 1 win / 1 tie / 3 losses.

The control we were missing in February

Same architecture, NO pretrained weights anywhere, classifier held fixed — so any difference is the objective



Masked pretraining earns its keep: +0.117 mean, positive on 5 of 5. This is the ONLY single-run result in the whole project that clears the noise floor.

Jigsaw and joint add nothing over a RANDOM projection. A random transformer is a strong baseline — so this means the objective, not the architecture, is the problem.

Part 2

Three things we got wrong

and the one experiment that caught them all

1

A control that did not control

we read an ablation backwards

2

A backwards hypothesis

finer patches = an easier task

3

An effect that never existed

and it was our headline result

Mistake 1 — a control that did not control anything

We had an ablation we believed showed 'pretraining does not help'

What we thought it tested

"Pretrained backbone vs random backbone." The numbers looked similar, so the reading was: pretraining is not buying us anything.

What it actually tested

It re-initialised only the layers that were being UNFROZEN for fine-tuning, leaving the rest pretrained — and the head was underfit in both arms, which masked the difference.

The fix

Build a genuine control: load NO pretrained weights anywhere (patch embedding and positional encoding included), and hold the classifier fixed at a converged LogReg in both arms. That is the experiment on the previous slide — and read correctly, masked pretraining HELPS by +0.117.

Lesson

A control has to be checked as carefully as the thing it controls. We now verify explicitly that a random-init arm produces genuinely different embeddings before trusting it.

Mistake 2 — a hypothesis that was exactly backwards

Our best explanation for the representation ceiling, tested properly and refuted

The hypothesis (very reasonable)

The backbone turns 131,072 points into 128 tokens, so it cannot represent detail finer than 128 positions. Classical LogReg uses 1,024 bins — 8× finer. So: shrink the patch, gain resolution.

Backbone	Barth	MTBLS326	BrC cancer	BrC diab	MTBLS563	mean Δ
patch 1024 (128 tokens)	0.806	1.000	0.859	0.780	0.618	baseline
patch 256 (512 tokens)	0.598	0.907	0.832	0.783	0.581	-0.072
patch 128 (1024 tokens)	0.655	0.911	0.768	0.738	0.607	-0.077

Finer patches won 0 of 5 targets — both arms were worse.

Why it failed — and this is the interesting part

Validation reconstruction loss FELL as patches shrank: $9.3 \times 10^{-5} \rightarrow 5.6 \times 10^{-5} \rightarrow 4.4 \times 10^{-5}$. A masked 128-point patch is largely interpolable from its neighbours, so shrinking the patch made the pretext task EASIER, not more informative — the model can solve it by local smoothing without learning any metabolite structure. Patch size and mask ratio jointly set task difficulty; we moved the wrong knob.

Mistake 3 — the big one. An effect that was never there.

How a single number became the largest reported result in the project

1 July

A new experiment needed a v4-pretrained baseline. We trained one.

2 The observation

It scored 0.820 held-out. The v3 checkpoint every earlier number used scored 0.888. Same architecture, same hyperparameters, config verified byte-identical.

3 The conclusion we drew

"The pretraining corpus version is worth +0.069 — larger than every other effect we have measured." We even recommended reverting to v3.

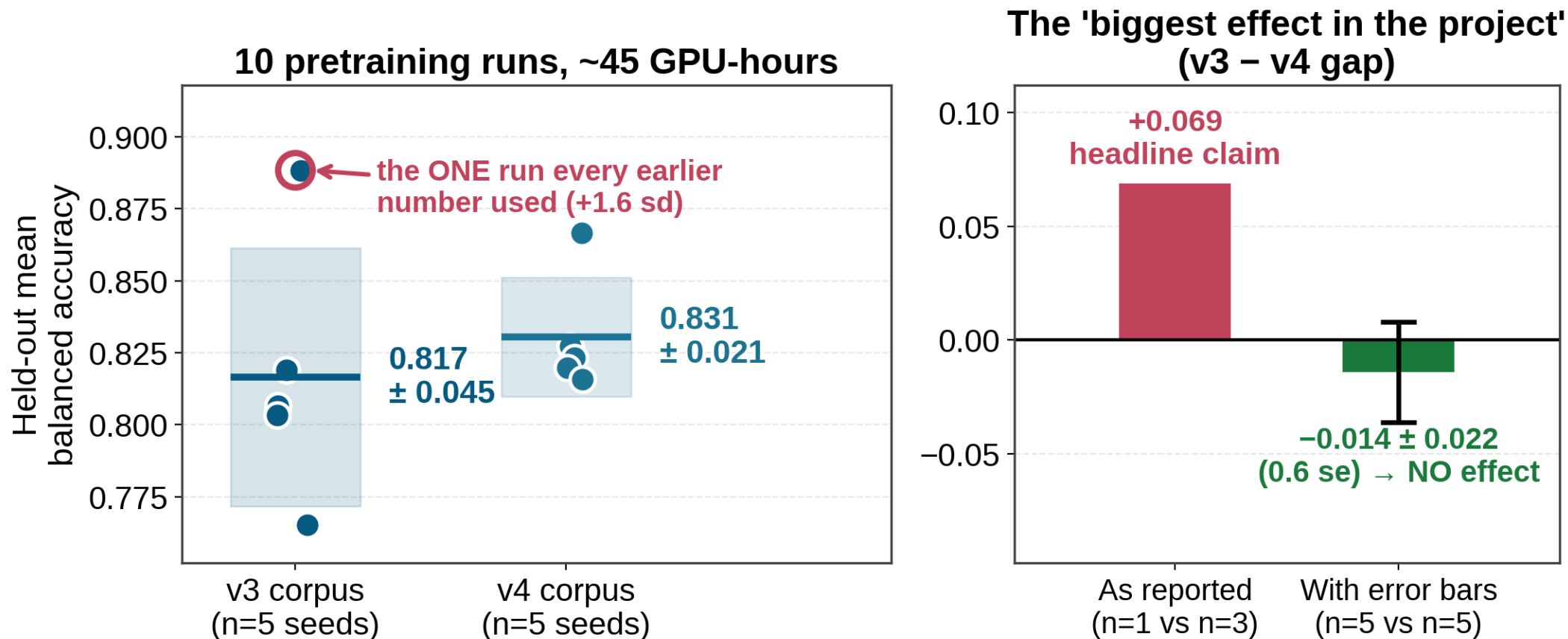
4 What we did next

Two more experiments (#8, #13) hunting the MECHANISM: was it the 164 differing rows? Was it corpus size? Both came back inconclusive.

Nobody asked the obvious question: how much does the SAME configuration vary between two training runs?

Experiment #15 — so we finally measured the noise

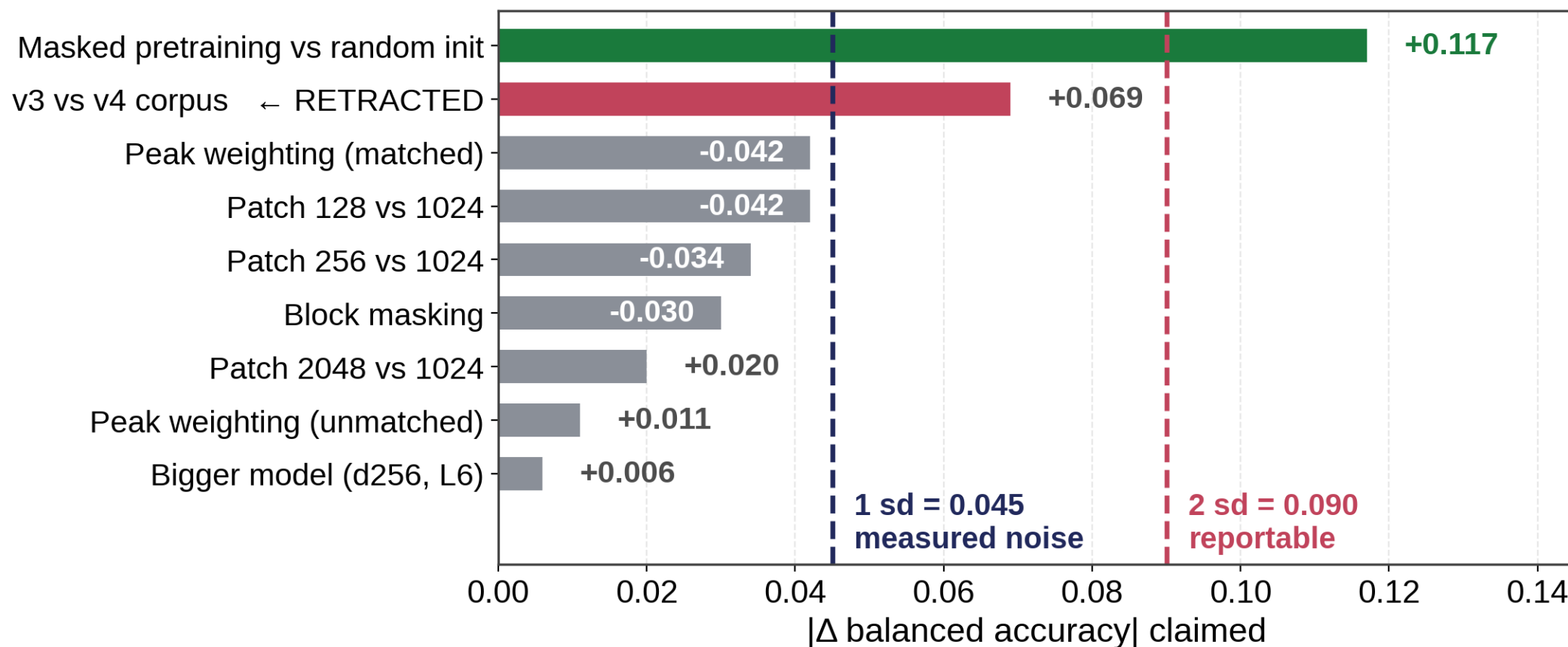
Ten pretraining runs, five per corpus, identical config differing only by random seed



The 0.888 reference was rank 1 of 5 and sits +1.6 sd above its own distribution. The gap does not shrink — it vanishes and marginally reverses. §5f retracted; experiments #8 and #13 were hunting the mechanism of a non-effect.

The worse news — the noise floor was 2× what we assumed

Every single-run claim in the project, re-scored against the measured spread



Measured sd = 0.045 on the held-out mean; per target up to 0.076 (Barth). We had been using 0.020 — which came from three runs whose errors happened to cancel.

New standing rule: no single-run difference below 0.09 is an effect. Either ≥ 5 replicates (~ 23 GPU-h) or a paired comparison — budgeted up front.

A fourth, more mundane failure — and the guardrails

Three wrong numbers in this project came from the same cause

Scoring unfinished checkpoints

Training writes a "best so far" checkpoint continuously. Evaluate one mid-run and you get a real-looking number for a model that was still training. It happened three times — including once when I claimed --seed was broken on GPU, because I compared a mid-training checkpoint against a finished one.

Also worth knowing

--seed DOES make training reproducible — two same-seed runs are byte-identical, $\max|\Delta W| = 0$. That earlier claim is retracted. Reproducibility was never the problem; run-to-run variance across DIFFERENT seeds was.

Provenance stamp

Checkpoints are written with `finished: false` and only flipped to true after the training loop exits cleanly.

Evaluator refuses

The probe evaluator hard-errors on an unfinished checkpoint and warns on legacy ones with no flag.

Loud failure on missing data

Analysis scripts now refuse a results directory missing an expected arm, instead of silently dropping it.

Every retraction in this project traces to one of two habits: comparing separately-trained networks, or trusting a checkpoint we had not verified was finished.

Part 3

Testing the premise directly

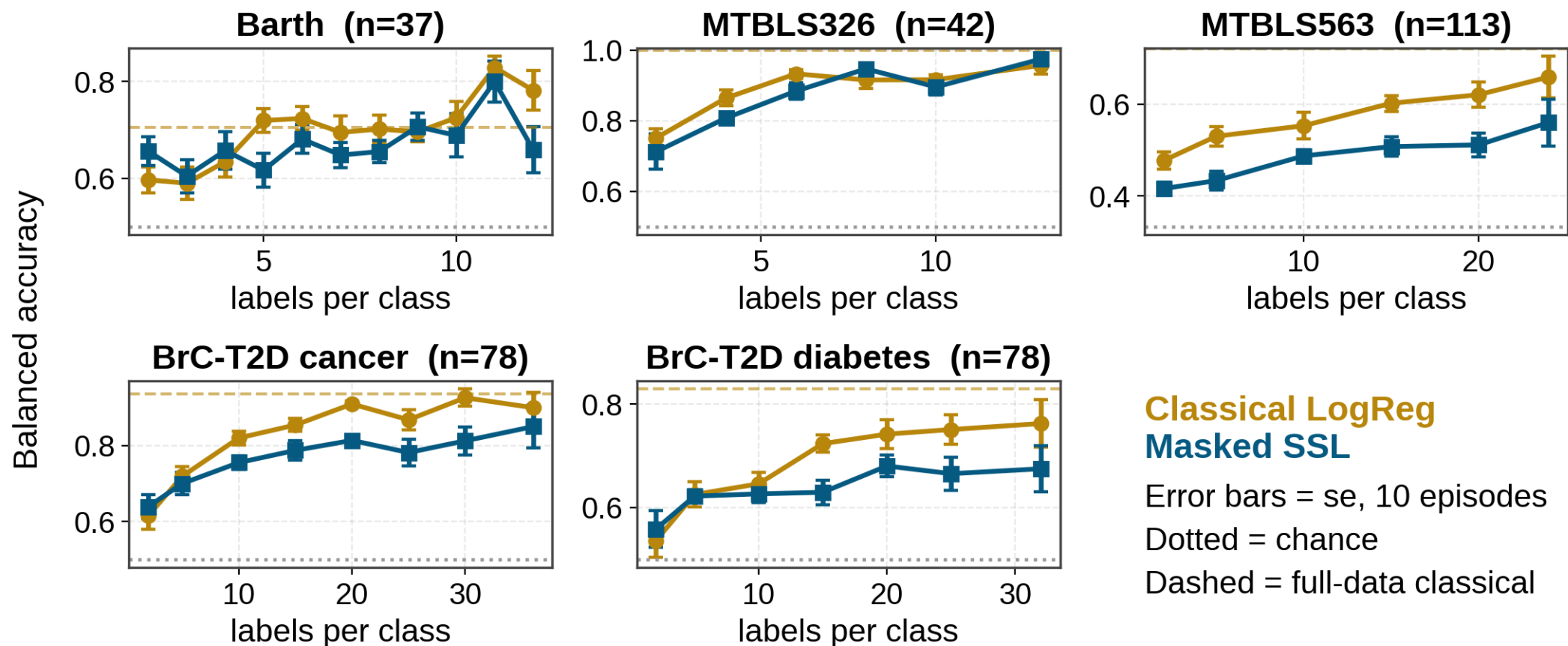
If pretraining helps anywhere, it must help when labels are scarce

The argument for doing this

Everything so far used the FULL dataset. At $n=37-113$ that is already close to the ceiling of what is learnable — the worst possible regime to look for a transfer advantage. Transfer is supposed to pay off with few labels. So: sweep the label budget from 2 per class upwards, and find where the SSL curve is above the classical one.

Experiment #6 — few-shot benchmark, all 5 targets

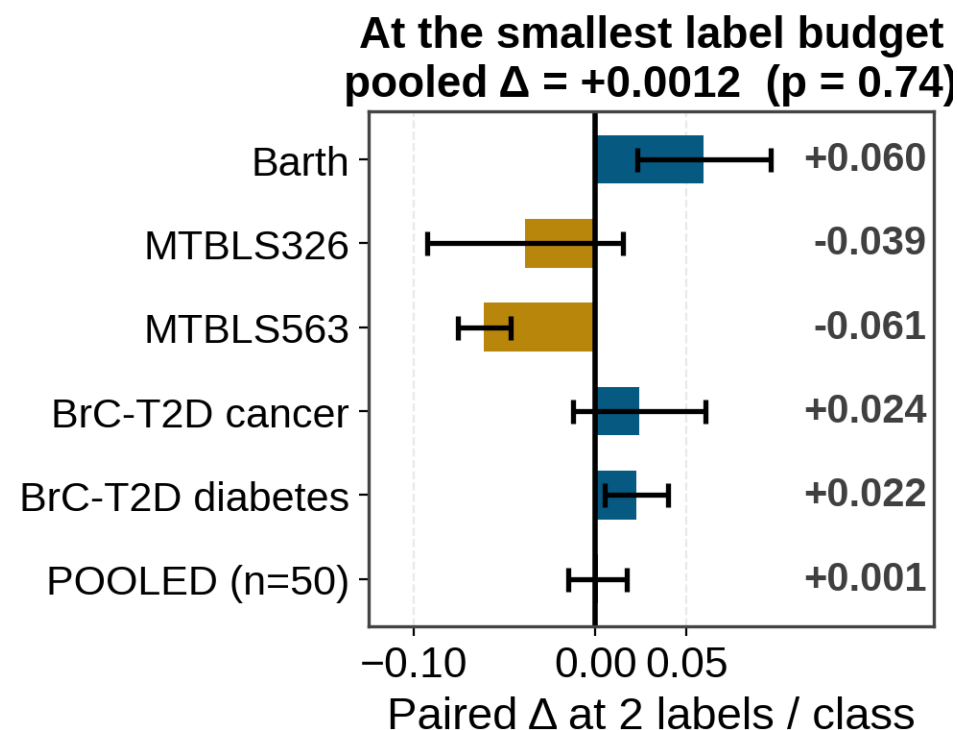
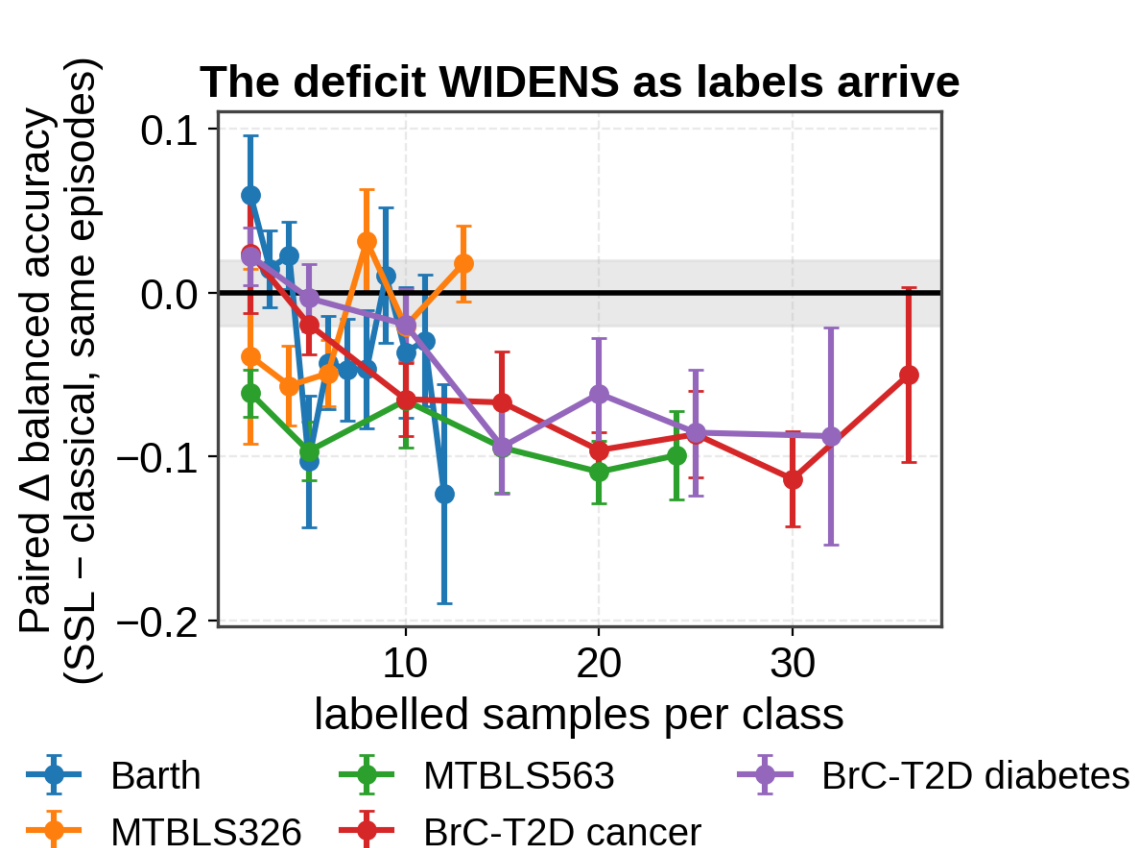
Every model sees the IDENTICAL support/query draws — 10 episodes per label budget



Masked SSL sits at or below classical in every panel, at every label budget. The premise was that SSL wins on the LEFT of these plots — there is no regime where the pretrained backbone wins.

The same data, analysed as a PAIRED comparison

Shared episodes mean the episode-draw variance cancels — far more power than comparing two error bars



Pooled across all five targets at 2 labels/class: $+0.0012 \pm 0.0162$, $p = 0.74$. Dead even. What looked like a low-shot edge is just both methods sitting near chance.

And the deficit WIDENS with more labels (negative trend on 4 of 5) — the opposite of what transfer learning predicts.

Before believing a negative result, we tried to break it

The obvious objection, and what happened when we tested it

The objection

Our pooling default (regional, G=16) gives 2,048 features. At 2 labels per class that is 2,048 features for 4 samples. Maybe we handicapped SSL precisely where we claim it has no advantage — a lower-dimensional pooling might win.

Target	labels /class	mean-pool (128 feat)	regional G=4 (512 feat)	regional G=16 (2048 feat)
Barth	2	0.609	0.627	0.656
Barth	5	0.564	0.603	0.617
BrC cancer	2	0.543	0.590	0.639
BrC cancer	5	0.597	0.644	0.700
MTBLS563	2	0.338	0.390	0.416
MTBLS563	5	0.338	0.387	0.433

The check resolves in favour of the conclusion — G=16 wins 6 of 6

Lower-dimensional poolings are WORSE at every low-support point tested. Even letting SSL pick its best pooling post-hoc per point — which inflates SSL — the pooled result is -0.033 ± 0.014 , $p = 0.006$. Still negative.

Also validated: the top of every few-shot curve lands just below that target's known full-data value (Barth 0.782 vs 0.705, cancer 0.900 vs 0.937, MTBLS326 0.957 vs 1.000). The harness measures what it should.

Where the project stands today

Everything that survived, and everything that did not

STILL STANDING

paired comparisons, plus the one 2 sd result

Masked pretraining > random init **+0.117**
2.6 sd, 5/5 targets

LogReg probe > trained DNN head **+0.120**
paired, 5/5

Position-preserving pooling **+0.03...+0.13**
paired, 5/5

Same pooling on jigsaw / joint **+0.079 / +0.049**
paired

RETRACTED or WITHIN NOISE

everything that rested on a single training run

v3 corpus better than v4 **+0.069**
n=5 → -0.014 (0.6 se)

"Backbone scaling exhausted" **±0.02**
within noise

Patch size 128 / 256 hurt **-0.04**
within noise

Block masking / peak weighting **-0.03 / +0.01**
within noise

"SSL wins in few-shot" **+0.001**
p = 0.74 — refuted

The pattern: every surviving result is **PAIRED** — one fixed checkpoint, one thing varied. Every retracted result compared two separately-trained networks.

What I think this means

Reading the whole picture honestly

The findings are coherent, not contradictory

Masked pretraining DOES learn something real (+0.117 over random init). But it learns less discriminative signal than 1,024-bin integrated areas already carry — and reconstruction quality anti-correlates with transfer. Meanwhile every axis we pushed on the backbone (patch size, capacity, objective, corpus version) came back flat.

The most likely cause we have NOT tested

The corpus is 9,670 spectra. That is very small for a foundation model — orders of magnitude below where masked pretraining usually starts paying off. Experiment #13 only probed corpus size DOWNWARD, which cannot detect an under-training regime.

The contribution, reframed

This is now a rigorous negative result on 1D NMR SSL: five targets, paired tests, a measured noise floor, replicated. The methodology — the 0.045 floor, the paired protocol, the guardrails — is arguably a stronger contribution than a backbone that didn't beat logistic regression.

What I would NOT recommend

More architecture variants. We have tried four patch sizes, two capacities, three objectives and two corpora; with the noise floor now known, none of those experiments could have detected an effect smaller than 0.09 anyway. Any further single-run backbone tweak is not worth the GPU time.

Proposed next steps

Ranked by what would actually change the answer

1

Cross-cohort transfer — the strongest untested case

Every evaluation so far is WITHIN a cohort. A foundation model's real selling point is generalising across instruments, sites and batches — exactly where absolute binned intensities should break down and a learned representation might not. Train the probe on one cohort, test on another. Cheap; we have 4 cohorts. If SSL wins anywhere, it is here.

2

Corpus scaling curve — upward this time

Pretrain at 25 / 50 / 100% of available spectra and see whether downstream utility is still climbing at 9,670. Flat $\Rightarrow$ the objective is the problem. Rising $\Rightarrow$ the answer is "needs 10–100 $\times$ more data". Either is publishable and actionable. Needs ≥ 5 seeds per point (~ 23 GPU-h each).

3

MTBLS326 batch-confound audit

Still outstanding, cheap, and MTBLS326 is one of only two targets where SSL is competitive. Its perfect 1.000 on $n=42$ is unverified as biology rather than run-order artifact.

4

Finish jigsaw / joint few-shot arms

Running now. The random-init control already predicts they will be worse than masking, so this is completeness rather than discovery.

Discussion

Three things I would like the group's view on

1

Is cross-cohort transfer the right last test?

It is the one setting where a learned representation should beat absolute binned intensities. If it fails there too, I would call the backbone approach closed for this data scale.

2

How much more pretraining data can we realistically get?

If we cannot get to ~100k spectra, the scaling experiment tells us the ceiling but not how to beat it. Are there cohorts or repositories we are not using?

3

Is a rigorous negative result publishable from this group?

Five targets, paired protocol, measured noise floor, three self-retractions. I think it is useful to the field — but I would like your read before I write it that way.

Thank you — and thanks to Gayatree for the Tirupati TBI samples that started this off.